

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

Protocolos de Comunicación: MCP y A2A

Módulo 10 · Sistemas Agénticos y Multi-Agente

MCP para herramientas, A2A para agentes, MCP Gateways, ecosistema 2025 y seguridad en producción.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Protocolos de Comunicación: MCP y A2A

A medida que los sistemas agénticos crecen en complejidad y escala, emerge un problema de integración: cada agente necesita conectarse a múltiples herramientas y sistemas externos (bases de datos, APIs, sistemas empresariales), y los agentes de diferentes frameworks necesitan comunicarse entre sí. Sin protocolos estándar, cada integración requiere código personalizado: el problema $N \times M$ donde N agentes necesitan M conectores para M sistemas.

En 2024-2025, dos protocolos han emergido para resolver estos problemas: MCP (Model Context Protocol, Anthropic noviembre 2024) para la comunicación entre agentes y herramientas, y A2A (Agent-to-Agent, Google abril 2025) para la comunicación entre agentes de diferentes sistemas. La adopción ha sido extraordinariamente rápida: para mediados de 2025, todos los grandes labs (OpenAI, Google, Microsoft, AWS) habían anunciado soporte para MCP.

Model Context Protocol (MCP)

El problema que resuelve MCP

Antes de MCP, conectar un agente LLM a N sistemas externos requería N implementaciones personalizadas: un conector para Salesforce, otro para Google Drive, otro para GitHub, etc. Con $N=50$ sistemas, son 50 integraciones personalizadas que mantener. MCP resuelve el problema $N \times M$ reduciéndolo a $N+M$: N agentes implementan el cliente MCP una vez, M sistemas implementan el servidor MCP una vez, y cualquier agente puede usar cualquier sistema sin código adicional.

La arquitectura de MCP

MCP sigue una arquitectura cliente-servidor. El servidor MCP expone las capacidades de un sistema a través de tres primitivas estándar: Tools (funciones que el agente puede llamar, análogas al function calling), Resources (datos que el agente puede leer, como archivos o registros de base de datos), y Prompts (templates de prompts predefinidos para tareas comunes con ese sistema). El cliente MCP (el agente) descubre las herramientas disponibles consultando al servidor y las usa de la misma forma que usa function calling nativo.

A2A: comunicación entre agentes

A2A (Google, abril 2025) es el complemento de MCP para la comunicación entre agentes. MCP define cómo un agente se conecta a herramientas; A2A define cómo un agente se comunica con otro agente de un framework diferente. Un agente LangGraph puede delegar una subtarea a un agente CrewAI a través de A2A sin conocer los detalles internos del agente receptor. A2A usa una interfaz estandarizada de tareas (task submission, status polling, result retrieval) que cualquier framework puede implementar.

Implementación práctica de MCP

Para usar un servidor MCP con Claude (API): define la lista de servidores MCP a los que el agente tiene acceso en el parámetro `mcp_servers` de la llamada al API. El servidor MCP puede ser: un proceso local (`stdio transport` para servidores en el mismo proceso), un servicio remoto (`HTTP/SSE transport` para servidores en internet), o un servidor en la red local (para sistemas empresariales internos). Claude descubre automáticamente las herramientas expuestas por cada servidor MCP y puede usarlas con la misma sintaxis que las tools definidas en el código.

Para crear un servidor MCP propio con el SDK de Python (`pip install mcp`): define las herramientas con el decorador `@server.call_tool()`, los resources con `@server.read_resource()`, y los prompts con `@server.get_prompt()`. El servidor se inicia como un proceso independiente que el cliente MCP (el agente) puede conectar. El servidor MCP puede envolver cualquier API existente, base de datos, o sistema empresarial en la interfaz estándar MCP.

MCP Gateways para producción

En producción, los sistemas agénticos con múltiples servidores MCP necesitan un MCP Gateway que centralice la autenticación, el rate limiting, el logging de auditoría, y el control de acceso. Los gateways más importantes en 2025 son: Bifrost (open source, Go, 11 microsegundos de latencia, soporte para LLM routing + MCP governance en una plataforma), AWS API Management y Azure APIM con soporte MCP, y Kong con plugin MCP. El gateway es imprescindible para producción con múltiples agentes y múltiples servidores MCP.

SECCIÓN 4 · EJEMPLOS REALES

- MCP en un asistente de engineering (empresa de software): el asistente tiene acceso a servidores MCP para GitHub (leer/crear issues, PRs), Jira (gestión de tickets), Confluence (documentación), y la base de datos de producción (solo lectura). Cada integración tardó 1-2 horas de configuración del servidor MCP vs semanas de código personalizado anterior. El agente puede hacer desde crear un PR hasta actualizar la documentación técnica en un solo workflow.
- A2A para orquestación cross-framework en empresa multinacional: el sistema de IA de la división US (LangGraph) delega análisis de contratos al sistema de la división EU (CrewAI especializado en derecho europeo) a través de A2A. Ninguno de los dos equipos necesita conocer la implementación interna del otro. La interfaz A2A estandarizada hace la integración limpia y mantenible.
- MCP Gateway de Bifrost para agentic system de inversión: el sistema de análisis de inversiones tiene acceso a 15 servidores MCP (Bloomberg, Reuters, SEC EDGAR, múltiples brokers, base de datos interna). Bifrost gestiona: autenticación por servidor MCP, rate limiting por usuario, logging de auditoría de cada herramienta usada (requerido por compliance financiero), y control de acceso (analistas junior no pueden acceder a algunos servidores).

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Implementar servidores MCP sin autenticación. Los servidores MCP exponen capacidades reales del sistema (leer/escribir bases de datos, enviar emails, crear tickets). Sin autenticación, cualquier agente que conozca la URL del servidor puede usar esas capacidades. Implementa siempre autenticación (API keys, OAuth) en los

servidores MCP, especialmente para acciones que modifican datos.

Error 2: Dar acceso a todos los servidores MCP a todos los agentes. El principio de mínimo privilegio aplica igual con MCP: un agente de soporte al cliente no necesita acceso al servidor MCP de la base de datos de producción. Configura el acceso de cada agente a solo los servidores MCP que necesita para su función.

Error 3: Confundir MCP con A2A. MCP es para la comunicación agente → herramienta/sistema. A2A es para la comunicación agente → agente. Son complementarios, no alternativos. Un sistema complejo puede necesitar ambos: MCP para que los agentes se conecten a sistemas externos, y A2A para que los agentes se delegen trabajo entre sí.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para empezar con MCP en un proyecto nuevo: 1) Identifica los sistemas externos que el agente necesita usar. 2) Busca en el directorio de MCP (mcp.anthropic.com) si ya existe un servidor MCP para esos sistemas. 3) Si existe, configura el servidor con las credenciales necesarias y añádelo a la lista de `mcp_servers` en tu llamada al API. 4) Si no existe, implementa un servidor MCP usando el SDK de Python o JavaScript con las operaciones necesarias. 5) Para producción, añade un MCP Gateway para autenticación centralizada y auditoría.

Para implementar A2A en un sistema multi-agente cross-framework: el agente caller define la tarea con el formato estándar A2A (`task_id`, `objective`, `context`, `expected_output`). Llama al endpoint A2A del agente receptor para submit la tarea. Sondea el estado de la tarea periódicamente hasta que esté completada (`status: completed`) o falle (`status: failed`). Recupera el resultado del endpoint de resultado. El agente receptor puede ser cualquier framework que implemente el protocolo A2A.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M10-T5 (Tool use): MCP estandariza cómo se exponen y usan las herramientas en sistemas agénticos.
- M10-T6 (Patrones de orquestación): A2A habilita la orquestación multi-agente cross-framework.
- M9-T3 (APIs de IA): MCP y A2A son protocolos de la capa de API para sistemas agénticos.

SECCIÓN 8 · RESUMEN EJECUTIVO

MCP (Model Context Protocol, Anthropic noviembre 2024) estandariza la conexión entre agentes y herramientas/sistemas externos, reduciendo el problema $N \times M$ a $N+M$. Primitivas: Tools, Resources, Prompts. Adoptado por OpenAI, Google, Microsoft, AWS. Miles de servidores disponibles para sistemas empresariales comunes. A2A (Google abril 2025) estandariza la comunicación entre agentes de diferentes frameworks con una interfaz de tareas (`submit`, `poll`, `retrieve`). Los MCP Gateways (Bifrost, AWS APIM) centralizan autenticación, rate limiting, y auditoría para producción. Implementar autenticación siempre en servidores MCP. Principio de mínimo privilegio: cada agente accede solo a los servidores MCP que necesita.